Convergence of Control Architectures to Escalation Under Bounded Context

Patrick D. McCarthy

Abstract

Top-down allocation—where a supervisory process directs subproblems to lower levels—is viable when the concurrent delegation count k fits within bounded active context C_n. We prove that as the action repertoire $|A|$ grows under bounded C_n, concurrent supervisory demands k grow because total domain knowledge $|K|$ grows with $|A|$, exposing more dependency chains to evidence revision. Any control architecture that remains viable must progressively shed supervisory tracking of routine actions, offloading them to autonomous lower levels that signal upward only on failure. This shedding *is* escalation. This monotone increase in the escalation fraction is not a design choice but a scaling consequence: maintaining top-down tracking incurs $O(k \cdot \alpha)$ context overhead, which fills C_n as capability grows, degrading inference quality for novel problems. The crossover occurs when $k \cdot \alpha \geq C_n/2$—for biological working memory, at 2–3 concurrent delegations. Beyond the crossover, the only recovery is to stop tracking routine actions, which is to adopt escalation for those actions. Hierarchical decomposition delays the crossover (each level sees only $O(b)$ subordinates) but does not eliminate it: the number of management levels grows with $|A|$, consuming encoding capacity. As hierarchical top-down becomes more autonomous at lower levels, it converges toward escalation. The pressure is one-directional: escalation never converges toward top-down, because it never introduces anticipatory delegation context. This result refines precision allocation models including the Free Energy Principle [10, 11] and the System M routing architecture of Dupoux, LeCun, and Malik [9].

1 Introduction

Every system that acts under uncertainty must solve a control problem: which problems receive active reasoning, and which are handled by automatic, pre-encoded responses? This question is distinct from the content of reasoning; it concerns the *architecture of attention*. Under bounded active context C_n—a hard limit on the number of propositions simultaneously available for inference, the formal analogue of Simon's bounded rationality [16, 17]—the control architecture is not a free parameter but a survival-critical design choice.

Two complementary mechanisms address this problem:

1. **Top-down allocation.** A supervisory process at the highest encoding level L_n monitors all subsystems, evaluates relevance, and routes problems to appropriate levels. This architecture is implicit in precision weighting under the Free Energy Principle [10, 11], where higher cortical levels set the gain on prediction errors at lower levels. It is explicit in the System M proposal of Dupoux, LeCun, and Malik [9], where a meta-controller directs when Systems A (observation) and B (action) engage. Kahneman's popular account of System 1 and System 2 [13] implicitly assumes top-down control: System 2 "decides" when to override System 1.

2. **Escalation.** Lower levels handle problems autonomously. Higher levels engage only when lower levels fail, that is, when uncertainty cannot be bounded at the current level. This architecture is implicit in Global Workspace Theory [3, 8], where local processors broadcast to the global workspace only when local resolution fails.

At small scale, when the entire action repertoire fits in active context, top-down allocation is viable and may be preferable: the supervisor can hold complete state with room to spare, and the benefit of optimal routing outweighs the modest overhead. Intelligence, however, requires $|A| \gg C_n$: an entity must know far more than it can hold in mind at once. This paper proves that as $|A|$ grows under bounded C_n, any viable control architecture converges toward escalation properties. The convergence is not a design preference but a scaling consequence: maintaining top-down tracking incurs context overhead that grows with capability, eventually filling C_n and degrading the very inference the supervisor is meant to optimize. As in [15], "necessary" here means necessary for persistence under competition—selective necessity, not logical impossibility of alternatives. The only recovery is to stop tracking routine actions—which is to adopt escalation for those actions.

The proof proceeds as follows. Section 2 introduces the encoding hierarchy $E(c, r)$ and the formal definitions required for the result. Section 3 defines the escalation policy: informed actions are handled at the lowest sufficient level. Section 4 proves the central result (Theorem 2): top-down allocation incurs $O(k \cdot \alpha)$ context cost, and any mechanism that keeps top-down viable at scale necessarily converges toward escalation. Section 5 compares the two architectures directly. Section 6 engages with prior work.

This paper is the second in a series. Paper 1 [15] established that any system accumulating knowledge under bounded context necessarily maintains graph structure. The present paper addresses how that structure is *controlled*: how problems are routed through the encoding hierarchy.

2 Formal Framework

We introduce the minimal definitions required for the main results. We inherit the world state space W, the notion of proposition, knowledge base K, uncertainty $U(w, K) = -\log P(w \mid K)$ [7], and bounded active context C_n from [15]. An *informed action* $a \in A$ is an action grounded in propositions from K; we write $K^{[a]} \subseteq K$ for the set of propositions that ground a.

Definition 1 (Encoding Hierarchy)**.** An *encoding hierarchy* is a continuous two-dimensional space $E(c, r)$ parameterized by:

- **Runtime cost** $c \geq 0$: the computational cost of accessing and modifying the encoding.
- **Reversibility** $r \in [0, 1]$: the ease with which the encoding can be revised. $r = 1$ is fully reversible; $r = 0$ is permanent.

Boundaries within this continuous space are *interfaces* $L_0, L_1, \ldots, L_n$. The number of interfaces and their positions are substrate-determined: biological systems, artificial systems, and institutional systems partition $E(c, r)$ differently.

Table 1 illustrates the hierarchy with representative interfaces. The precise number and character of levels varies by substrate; what is invariant is the monotonic relationship between runtime cost and reversibility.

Interface	Description	Runtime Cost	Reversibility
L_n	Active reasoning / working memory	O(compute)	Fully reversible ($r \approx 1$)
L_{n-1}	Learned heuristics / cached policies	$O(1)$ retrieval	Revisable with effort
$\vdots$	*substrate-dependent intermediate levels*		
L_1	Developmental encoding / early learning	Near-zero	Difficult to revise
L_0	Heritable / constitutional encoding	0	Permanent ($r \approx 0$)

Table 1: Representative encoding hierarchy. Cost increases and reversibility increases from L_0 to L_n. The number of interfaces is substrate-determined.

Definition 2 (Active Context Capacity and Relevance Density)**.** The *active context capacity* C_n is bounded: $|\mathrm{ctx}(L_n, t)| \leq C_n$. For an informed action a, the *relevance density* at L_n is:

$$\rho(L_n, a) = \frac{|K^{[a]}|}{|\mathrm{ctx}(L_n, t)|}$$

where $K^{[a]} \subseteq K$ is the knowledge relevant to a. $\rho = 1$ is maximum signal: active context holds exactly what a requires. $\rho \to 0$ is noise-dominated: context is filled with content irrelevant to a.

Definition 3 (Delegation Overhead)**.** Under *top-down allocation*, L_n actively manages lower-level processes. For each active delegation, L_n must hold in context:

(i) The *subproblem specification*: what was delegated.

(ii) The *expected return signal*: what a successful result looks like.

(iii) The *integration context*: how the result will be incorporated into the current reasoning state.

Let $\alpha > 0$ be the *minimum context cost per delegation*: the irreducible number of context slots consumed by maintaining one active delegation. $\alpha > 0$ because each delegation requires at least one proposition specifying what was delegated.

3 Escalation

Definition 4 (Escalation Policy)**.** Under the encoding hierarchy (Definition 1), the *escalation policy* handles each action a at the lowest level L_i such that the knowledge encoded at L_i is sufficient to bound uncertainty for a. Escalation to L_{i+1} occurs if and only if L_i fails to bound uncertainty.

Observation. The escalation policy is cost-minimizing under monotone cost. Since $c(L_{i+1}) > c(L_i)$ by Definition 1, handling a at any level higher than the lowest sufficient level pays strictly more for the same uncertainty reduction. If $j = n$, handling a at L_n also consumes bounded active context, reducing ρ for the next novel problem. The escalation policy is strictly dominant on both cost and context conservation. This is immediate from the monotone cost structure and requires no proof.

The substantive question is not whether escalation is cost-minimizing —it is, by construction— but whether bounded systems *converge toward* escalation as they scale. That is the content of Theorem 3.

Remark. The escalation policy does not claim that all systems *implement* escalation. Path dependence, architectural constraints, and design choices can produce systems that use higher levels than necessary. Such systems pay a strictly positive cost premium. The claim is normative (escalation is cost-optimal) rather than descriptive (all systems escalate).

Corollary 1 (Context Window as Frontier)**.** *L_n is the frontier of unbounded uncertainty, not the seat of intelligence. A problem reaching L_n is evidence of failure at every lower level. An entity that routes most problems through L_n is not "thinking more"; it is failing to encode.*

Remark (Free context). The quantity that escalation preserves is *free context*: $C_{\text{free}}(t) = C_n - |\text{ctx}_{\text{unresolved}}(t)|$, the active context capacity available for novel problems after existing commitments. Under top-down allocation, delegation overhead reduces C_{free} by $k \cdot \alpha$ (Theorem 3). Under escalation, routine actions consume zero L_n capacity, so C_{free} is bounded below only by the current novel problem. Escalation is the control architecture that maximizes C_{free}; this is the mechanism by which capability growth *relieves* rather than burdens the highest encoding level. Escalation protects C_{free}: graph structure creates it by compressing shared propositions, knowledge curation maintains it by displacing low-value propositions, and escalation shields it from supervisory overhead.

Corollary 2 (Simultaneous Multi-Level Activity)**.** *All levels of the hierarchy operate concurrently. L_0 handles heritable responses, L_1 handles developmental encodings, and intermediate levels handle their respective domains, simultaneously. L_n handles only the residual: problems that no lower level can resolve. The escalation architecture does not serialize processing through a hierarchy; it parallelizes across levels, with upward propagation occurring only on failure.*

4 Convergence to Escalation Under Bounded Context

Theorem 3 (Convergence to Escalation at Scale)**.** *Under top-down allocation, the supervisory level L_n must maintain delegation state for managed subprocesses (Definition 3). Let $k(t)$ be the number of concurrent delegations at time t. The relevance density for a novel problem a_{novel} is:*

$$\rho(L_n, a_{\text{novel}}) = \frac{|K^{[a_{\text{novel}}]}|}{|K^{[a_{\text{novel}}]}| + k(t) \cdot \alpha}$$

Assumption *(concurrent demand growth): $k(t) \to \infty$ as $|A| \to \infty$. Section 4.1 justifies this from the dependency structure of K. Given this assumption, as $|A|$ grows under bounded C_n:*

(i) $\rho \to 0$: inference quality degrades monotonically.

(ii) Recovery requires either growing C_n (impossible under boundedness) or reducing k—that is, shedding supervisory tracking of actions that lower levels can handle autonomously.

(iii) The shed actions are, by definition, operating under escalation: handled at the lowest sufficient level, with upward signaling only on failure.

Let $\phi(t) = |\{a \in A(t) : a \text{ operates under escalation}\}|/|A(t)|$ be the fraction of the action repertoire under escalation. Since at most $k^* = C_n/(2\alpha)$ actions can be under active top-down supervision simultaneously (Section 4.1),

$$\phi(t) \;\geq\; \frac{|A(t)| - k^*}{|A(t)|} \;=\; 1 - \frac{k^*}{|A(t)|}$$

As $|A| \to \infty$ under bounded C_n, $\phi \to 1$. The bound k^* is fixed by substrate (C_n and α are constants), so the escalation fraction approaches unity not because every action is shed but because the number of actions that can *remain under top-down supervision is bounded while the total repertoire grows without bound.*

Proof. Under top-down allocation, L_n serves two simultaneous functions: (a) reasoning about the novel problem a_{novel}, requiring $|K^{[a_{\text{novel}}]}|$ context slots, and (b) maintaining delegation state for $k(t)$ concurrently managed subprocesses, each consuming α context slots (Definition 3).

The total context consumed is:

$$|\text{ctx}(L_n, t)| = |K^{[a_{\text{novel}}]}| + k(t) \cdot \alpha$$

Since $\alpha > 0$, any growth in $k(t)$ increases the denominator while the numerator $|K^{[a_{\text{novel}}]}|$ is bounded per problem. This establishes Part (i): ρ decreases monotonically with $k(t)$.

Part (ii): the dilemma at scale. By premise, $k(t)$ grows with $|A|$. Under flat top-down, the supervisor tracks all active processes directly. Under hierarchical top-down with branching factor b, each supervisory level tracks b subordinates across $\lceil \log_b |A| \rceil$ levels. In both architectures, the supervisory load at L_n grows with the repertoire.

The critical threshold occurs when $k(t) \cdot \alpha \geq C_n - |K^{[a_{\text{novel}}]}|$: delegation overhead has consumed all context capacity not occupied by the current problem's knowledge. To maintain ρ at a fixed level, C_n must grow at least linearly with $k(t)$. But C_n is bounded by substrate. The only alternative is to reduce k—to stop tracking actions that lower levels can handle without supervision.

Part (iii): shedding tracking is escalation. A supervisor that delegates but does not track (fire-and-forget) holds no return-integration context and cannot evaluate, redirect, or integrate results; it is not supervising those actions. Those actions are self-managing: handled at the lowest sufficient level, with upward signaling only on failure. This is precisely the escalation policy (Definition 4).

The convergence follows: as $|A|$ grows, a viable system must shed top-down tracking of an increasing fraction of its repertoire. The shed fraction operates under escalation. The supervisor retains top-down control only for the residual that no lower level can handle—genuinely novel problems. The system has not abandoned top-down allocation; it has converged to a hybrid where escalation handles the routine and top-down handles the novel. This hybrid is the only architecture that remains viable under bounded C_n as $|A| \to \infty$. □

4.1 The crossover: when top-down yields to escalation

Top-down allocation is viable when delegation overhead fits comfortably within C_n. The crossover is defined in terms of *concurrent delegations* k, not total repertoire $|A|$. An entity with 10,000 learned actions does not supervise all 10,000 simultaneously; k is the number of subprocesses the supervisor is actively tracking at time t. The crossover occurs when:

$$k^* \cdot \alpha = \frac{C_n}{2} \quad \Longrightarrow \quad k^* = \frac{C_n}{2\alpha}$$

Below k^*, the supervisor retains at least $C_n/2$ slots for reasoning about novel problems. Above k^*, delegation overhead consumes the majority of C_n. The specific fraction $C_n/2$ is a convention, not a derived quantity; the qualitative result—a finite crossover exists—holds for any positive fraction. The quantitative estimate ($k^* = 2$–3 for humans) is sensitive to this choice.

Estimating the crossover. The minimum delegation cost α is at least 1: specifying "what was delegated" requires at least one proposition in context. Under Cowan's [5] revised estimate of working memory capacity ($C_n \approx 4$), the crossover is $k^* = 4/2 = 2$ concurrent delegations. Under Miller's more generous bound ($C_n \approx 7$), $k^* = 3.5$. A human supervisor can actively track 2–3 concurrent delegations before reasoning quality degrades. This aligns with the empirical observation that human "multitasking" degrades performance sharply beyond two concurrent tasks: the delegation overhead of tracking a third task consumes the context needed to reason about any of them.

Justification of the concurrent demand growth assumption. Theorem 3 assumes $k(t) \to \infty$ as $|A| \to \infty$. We justify this with a simple model of evidence cascades.

Let $|K|$ denote the number of propositions and $\bar{d}$ the average out-degree (number of dependents per proposition). Under positive novelty rate $\nu > 0$, evidence events arrive at rate ν, each hitting one proposition and cascading to its dependents. Each cascade produces $\bar{d}$ propositions requiring re-evaluation. If the mean resolution time per proposition is τ, the expected number of concurrently unresolved items is:

$$\mathbb{E}[k] \;=\; \nu \cdot \bar{d} \cdot \tau$$

A supervisor could batch multiple unresolved items into a single delegation ("handle all re-evaluations in domain X"). Batching reduces the number of tracked delegations but does not eliminate the overhead: specifying each batch still costs at least α per batch, and verifying that a batch was handled correctly requires loading its scope into context. Batching changes the constant in $k \cdot \alpha$ but not the scaling behavior as $|A| \to \infty$.

The key: $\bar{d}$ grows with $|K|$. As $|A|$ grows, overlapping action policies share more propositions, and those shared propositions are extracted into K (the normalization that any bounded system converges toward, since maintaining redundant copies exceeds C_n as the repertoire scales). Paper 1 [15] establishes that edge density increases as K matures (Edge Growth Corollary): $\bar{d}$ is monotonically non-decreasing in $|K|$. Since $|K|$ grows with $|A|$ and $\bar{d}$ grows with $|K|$, we have:

$$\mathbb{E}[k] \;=\; \nu \cdot \bar{d}(|K|) \cdot \tau \;\xrightarrow{|A| \to \infty}\; \infty$$

provided $\nu > 0$ and $\tau > 0$ (resolution is not instantaneous). The growth is driven by edge densification: a richer knowledge base has a larger surface area for evidence to hit, producing more concurrent unresolved items per unit time.

Under pure top-down, all concurrent unresolved items would require supervisory tracking, producing $k \gg k^*$. The only way to keep $k \leq k^*$ is to ensure that most concurrent demands are handled autonomously at lower levels—that is, under escalation. The larger $|A|$ grows, the larger the fraction that must operate under escalation to keep k bounded. This is the convergence of Theorem 3(iii): escalation is not adopted as a design choice but forced by the scaling arithmetic.

The crossover clarifies the scope of convergence. Top-down allocation is not wrong; it is viable at small scale, when concurrent demands fit within C_n. But intelligence requires $|A| \gg C_n$: an entity that can only act on what it can simultaneously hold in mind is not intelligent but reactive. Every physical system that we recognize as intelligent—from insects routing foraging with stigmergic signals to humans operating with ∼7 active context slots across thousands of learned skills—appears to have converged, consistent with the cost-minimization pressure of the escalation policy (Definition 4): the vast majority of its repertoire operates under escalation, with top-down engagement reserved for novel coordination.

Time-multiplexing converges to escalation. An objection: the supervisor need not hold all k delegations simultaneously. It could round-robin, checking one subordinate at a time, clearing

context between checks. This eliminates the simultaneous $k \cdot \alpha$ load. But consider what the supervisor must maintain across checks: a *delegation list*—which subordinates were assigned which tasks—so it knows whom to check next. This list is the delegation context; storing it costs $k \cdot \alpha$ whether held in active context or reloaded from long-term storage (with context-switch cost γ per reload). As k grows, the total cycle time $k \cdot (\alpha + \gamma)$ grows, and the supervisor falls behind evidence arrival (rate ν). The crossover still exists.

The only way to eliminate the delegation list entirely is to stop tracking what was delegated: the supervisor holds zero delegation state and processes signals *as they arrive from below*. But a supervisor with no delegation list that processes upward signals on arrival is not time-multiplexing top-down control—it is escalation (Section 5). Time-multiplexing does not avoid the convergence; it is a way station on the path toward it.

Cell division as illustration. A single-celled organism can "manage" its repertoire top-down: the entire state fits in one context. A multicellular organism with 10^{13} cells cannot manage each cell individually through a central controller.

Biology's actual solution is a *hybrid*: local autonomy (cells monitor nutrient gradients, contact inhibition, and growth factor concentrations) combined with systemic signaling (hormonal regulation, tumor suppressors, immune surveillance). The systemic signals are genuine top-down control, but they are *broadcast* signals (growth hormone affects all cells of the right type), not individual delegations (no central process tracks each cell's state). Broadcasting is $O(1)$ in delegation context: the signal is sent once regardless of the number of recipients. Individual delegation would require tracking 10^{13} cells, which is well beyond any bounded context.

The hybrid architecture is precisely the crossover result: top-down broadcast control for system-wide coordination (below the crossover for broadcast), combined with local escalation for individual cell decisions (above the crossover for per-cell management). When the systemic signaling fails, when a cell ignores growth suppression signals, the immune system engages: escalation on failure.

4.2 The hierarchical top-down objection

The strongest objection to Theorem 3 is that it attacks flat top-down allocation, a strawman. Real supervisory systems are *hierarchical*: a top level manages b intermediaries, each managing b sub-intermediaries, so that overhead at any single level is $O(b)$, not $O(|A|)$. Does hierarchical decomposition rescue top-down allocation?

No. The hierarchical decomposition is more subtle than the flat case, but it does not rescue top-down allocation. We address the strongest version of the objection directly.

(1) L_n sees only $O(b)$ overhead, but the hierarchy has a different cost. The strongest version of the hierarchical objection notes that L_n manages only b direct subordinates, so overhead at L_n is $b \cdot \alpha$, constant in $|A|$. The $\log_b |A|$ levels of overhead are distributed across intermediate levels, each with its own bounded context. No single level overflows. This is correct and is a genuine advantage of hierarchical decomposition over flat top-down.

However, the cost is not eliminated; it is *redistributed*. The hierarchy requires $\lceil \log_b |A| \rceil$ supervisory levels, each of which devotes $b \cdot \alpha$ of its context to delegation state. The number of management-dedicated levels grows with $|A|$.

(2) Management levels displace knowledge levels. This is the deeper cost, and it has a precise analogue in organizational economics. Williamson [19] proves that each management layer in a hierarchical organization introduces "control loss": a fraction $\alpha_W < 1$ of the intended directive

is lost at each level, so that n management levels transmit only α_W^n of the top-level intent. This limits the viable depth of hierarchical control and, consequently, organization size. The same structure applies here: each intermediary in a hierarchical top-down system is itself a supervisory process that must hold delegation context, and the cumulative cost of these management layers limits the encoding capacity available for knowledge.

In an escalation architecture, every encoding level is a *knowledge level*: it holds proven actions and propositions, not routing state. In hierarchical top-down, $\lceil \log_b |A| \rceil$ levels are *management levels*: they hold delegation state. As $|A|$ grows, the number of management levels grows (logarithmically), consuming encoding capacity that escalation would use for knowledge. Williamson's result shows this is not merely wasteful but structurally limiting: the management overhead places an upper bound on the effective depth of hierarchical control, just as it places an upper bound on firm size.

The cost is not context overflow at any single level but a growing fraction of the entity's encoding hierarchy devoted to management rather than knowledge. An entity with 10 encoding levels and $|A|$ requiring 4 management levels has only 6 levels for knowledge; escalation would use all 10. The management overhead is in *levels consumed*, not *context slots consumed at* L_n.

(3) The causal direction is the fundamental difference. The deepest distinction between hierarchical top-down and escalation is not the cost asymptotic but the *causal direction of engagement*:

- In hierarchical top-down, the upper level *initiated* the delegation. It is waiting for a result it requested. It must hold return-integration context: what was delegated, what a successful result looks like, and how to incorporate it. This context is held *whether or not the subprocess encounters difficulty.* A supervisor waiting for ten reports holds ten slots regardless of whether any report is late.
- In escalation, the upper level *did not initiate* anything. It holds zero context for problems handled below. It loads context only when a failure signal arrives from below, and in steady state, failures are rare because lower levels handle most problems. There is nothing to "wait for" because nothing was requested.

This is not a quantitative difference in overhead; it is a structural difference in what context is held and why. Hierarchical top-down holds *anticipatory context* (waiting for results that may or may not arrive). Escalation holds *reactive context* (responding to failures that have already occurred). Anticipatory context scales with the number of delegations; reactive context scales with the failure rate. As the system matures and lower levels improve, the failure rate decreases, so escalation overhead *decreases with capability.* In hierarchical top-down, the number of delegations *increases with capability*: more capability means more subprocesses, each requiring a delegation slot.

Convergence of hierarchical top-down. This is the central point. Hierarchical top-down that is sufficiently deep, with intermediaries that handle most problems without escalating, begins to *resemble* escalation. The intermediary that resolves a problem without reporting upward is functioning as an escalation level: it holds no delegation context at the level above. The convergence is one-directional: as hierarchical top-down becomes more autonomous at lower levels, it converges toward escalation. Escalation never converges toward top-down, because it never introduces anticipatory delegation context. Hierarchical decomposition is not an alternative to escalation; it is the *intermediate regime* through which top-down converges toward escalation as $|A|$ grows.

4.3 Event-driven "top-down" and causal direction

A fourth objection: an event-driven top-down supervisor registers callbacks, holds zero delegation state, and is invoked by subsystems on completion or failure. This achieves escalation's $O(1)$ steady-state cost while retaining "top-down authority."

The issue is not one of labeling; it is not that we absorb every efficient architecture under the name "escalation" to make the thesis unfalsifiable. The issue is the *causal direction of the signal*. The structural distinction between top-down and escalation is:

- **Top-down**: the upper level initiates. The signal goes *downward*. The upper level says "do this" and waits for the result.
- **Escalation**: the lower level signals. The signal goes *upward*. The lower level says "I cannot handle this" and the upper level engages reactively.

In the event-driven architecture, who initiates the callback? The *subsystem*. The subsystem detected the condition, whether completion or failure, and sent the signal *upward*. The upper level did not poll, did not check, did not anticipate. It was invoked from below. This is bottom-up signaling regardless of what the architecture is called.

The claimed top-down authority, the ability to initiate, cancel, or redirect, is unavailable to a supervisor holding zero state. A supervisor with no delegation context does not know what is happening below and cannot redirect what it is not tracking. When the callback arrives, the supervisor must *reload context* to interpret the result, exactly what an escalation level does when a failure signal arrives.

The point is not definitional but empirical: if you trace the causal arrow in the event-driven architecture, it points upward. The architecture may have been *designed* as top-down (the supervisor originally assigned the task), but in steady-state operation, the information flow is bottom-up. The label "top-down" describes the initial assignment; the ongoing control structure is escalation.

4.4 Hidden costs of escalation during learning

A second objection: escalation achieves $O(1)$ steady-state cost, but this assumes lower levels are competent. During learning, when lower levels frequently fail, escalation produces frequent failure signals at L_n, burdening it with escalation context.

This is correct but not fatal to the argument. During learning, both architectures burden L_n: top-down burdened by delegation overhead plus the novel problems themselves; escalation burdened only by the novel problems (the failures that escalate). The difference: under top-down, the overhead is *additive* to the learning load (delegation context plus problem context). Under escalation, the overhead *is* the learning load (failure context only). Escalation during learning is not $O(1)$; it is $O(\text{failure rate} \cdot |K^{[a]}|)$, but it never exceeds the intrinsic difficulty of the problems themselves. Top-down adds coordination overhead on top of intrinsic difficulty.

As learning proceeds and lower levels encode proven actions, escalation load decreases monotonically. Top-down delegation overhead does not decrease with learning; it grows with the number of managed processes, which increases as the system becomes more capable.

4.5 Empirical success of dense attention

A third objection: transformers with full self-attention, the purest top-down architecture, where every token attends to every other, have dominated alternative architectures for a decade. If top-down truly degraded, why does dense attention win?

The answer is in the trajectory, not the snapshot. Dense attention succeeds *today* at *current scales*. But the trend is toward escalation-like structure:

- **Sparse attention** (learned sparsity patterns, local windows, routing) reduces $O(n^2)$ by learning which edges to keep—learned graph pruning [15].
- **Mixture-of-experts** routes tokens to specialized sub-networks rather than evaluating all parameters for all tokens: escalation at the parameter level.
- **Retrieval-augmented generation** retrieves relevant context from an external store rather than holding everything in the context window: graph traversal from a query node.
- **"Lost in the middle"** [14]: empirical evidence that inference quality degrades as context fills with irrelevant content: precisely the ρ degradation of Theorem 3.

The industry is converging toward hybrid architectures that incorporate escalation. Dense attention is the starting point, not the destination. The cost pressures the theorem identifies are driving the convergence.

Remark: the LLM theory of the case. Theorem 3 does not claim that large language models *cannot work*. Dense attention over a large context window is a viable strategy as long as the context can cover the problem space, that is, as long as the system operates below the crossover (Section 4.1). Current LLMs succeed precisely because their context windows are large enough, relative to the problems they are asked to solve, that ρ degradation is tolerable.

What the theorem establishes is that this strategy is *suboptimal and expensive*. Full self-attention recomputes the pairwise adjacency structure from scratch at every query: $O(n^2)$ per inference step. A system with stored adjacency pays $O(d)$ per retrieval, where d is the local degree. The cost ratio grows quadratically with context size. For a 128k-token context window, the system computes $\sim 10^{10}$ pairwise attention weights per layer per query, the vast majority of which are near-zero (irrelevant pairs). This is the computational cost of not having a graph.

The theorem also predicts a ceiling. As the problems LLMs are asked to solve grow in complexity, requiring more knowledge, more reasoning steps, more integration across domains, the context required grows with the problem, and ρ degrades: "lost in the middle" [14] is the empirical manifestation. Growing the context window defers the crossover but does not eliminate it, because the overhead of attending to irrelevant content grows quadratically while the benefit of additional context grows sub-linearly (most added context is irrelevant to any specific problem).

The LLM architecture is not invalidated by this result; it is *located* by it. LLMs operate in the pre-crossover regime where dense top-down attention is functional but costly. The industry trajectory toward sparse attention, mixture-of-experts, and retrieval-augmented generation is movement toward the post-crossover architecture the theorem identifies as necessary: stored adjacency, selective retrieval, escalation on failure.

Remark on scope. Theorem 3 does not claim that top-down control is unnecessary. Top-down allocation handles genuinely novel problems that no lower level can resolve, and occasional top-down intervention, where L_n deliberately overrides a lower-level response, is a valid and sometimes

necessary operation. The theorem establishes that top-down allocation as the *sole* control architecture is unsustainable under bounded C_n. Escalation is what preserves the context capacity that top-down allocation needs. The distinction is between top-down as the exclusive routing mechanism (unsustainable under bounded context) and top-down as the highest-level component in a complementary architecture where escalation handles the routine.

Biological measurement. The empirical signature of escalation is the transition from active to automatic processing across skill acquisition. When an informed action a is first learned, it consumes L_n capacity: the learner must actively reason through each step. As encoding proceeds, as $K^{[a]}$ migrates from L_n to lower levels, a ceases to consume L_n capacity. This is precisely the escalation architecture: a has been encoded at a level where it no longer requires active context. The ubiquity of this transition across motor learning, language acquisition, and expert skill development [13] is evidence that biological systems implement escalation rather than persistent top-down allocation.

5 Comparison: Escalation vs. Top-Down

The two architectures can be compared directly on their steady-state context demands.

5.1 Escalation architecture

Under escalation, L_n engages only when lower levels fail to bound uncertainty for a given problem.

- **Steady-state context per problem:** $O(|K^{[a]}|)$. Only the knowledge relevant to the currently escalated problem occupies L_n. No delegation overhead is incurred because lower levels are not "managed"; they operate autonomously.
- **Effect of knowledge growth:** As K grows and more actions are encoded at lower levels, *fewer* problems reach L_n. The load on active context decreases or remains bounded as capability increases.
- **Scaling property:** Capability growth *relieves* L_n rather than burdening it. Each successfully encoded action is one fewer demand on active context.

5.2 Top-down allocation architecture

Under top-down allocation, L_n holds delegation context for all managed subprocesses.

- **Steady-state context per problem:** $O(|K^{[a]}| + k \cdot \alpha)$, where k is the number of concurrent delegations. The delegation overhead $k \cdot \alpha$ is additive to the problem's own knowledge requirements.
- **Effect of capability growth:** As A grows, k grows proportionally (the supervisor must track more subprocesses). L_n load increases with capability.
- **Scaling property:** Capability growth *burdens* L_n. Each new capability adds to the supervisory overhead, eventually saturating active context.

5.3 The inversion

The escalation architecture inverts the scaling relationship between capability and cognitive load:

	Escalation	**Top-Down**
L_n engagement	On failure only	Continuous supervision
Steady-state context	$O(\|K^{[a]}\|)$	$O(\|K^{[a]}\| + k \cdot \alpha)$
As $\|A\|$ grows	L_n load $\downarrow$ or bounded	L_n load $\uparrow$ proportionally
At capacity	Novel problems get full C_n	Novel problems get $C_n - k\alpha$

Under escalation, a system that has encoded 10,000 actions at lower levels has *more* active context available for novel problems than when it had encoded 100, because 9,900 additional problems are now handled without L_n engagement. Under top-down allocation, a system managing 10,000 subprocesses has *less* active context available than when it managed 100, because 9,900 additional delegation slots consume context.

This is not a quantitative difference but a qualitative inversion of the scaling direction. It explains why expertise feels like *effortlessness*: the expert has encoded actions that the novice must actively reason through, freeing L_n for genuinely novel problems.

6 Engagement with Prior Work

6.1 Free Energy Principle and Active Inference

The Free Energy Principle [10, 11] posits that biological systems minimize variational free energy, with attention implemented as precision weighting: higher cortical levels set the gain on prediction errors at lower levels. Precision weighting is not top-down delegation in the sense of Definition 3—it does not track individual subproblems with return-integration context. It is a gain modulation mechanism: a higher level sets a parameter on a lower-level channel. The cost per channel is lighter than full delegation overhead.

The convergence result still applies, because precision weighting has a nonzero cost per channel. Each precision parameter the higher level maintains is a representation it must hold, update, and evaluate against incoming prediction errors. At small scale, when the number of channels k is modest, this cost fits comfortably within C_n and precision weighting is viable—potentially optimal. As the system scales and k grows, maintaining precision parameters for all channels consumes an increasing fraction of C_n. The system must shed active precision tracking for routine channels: channels whose prediction errors are consistently small no longer need higher-level gain modulation and can operate at locally set precision. This shedding is the same convergence Theorem 3 identifies: at scale, the system retains active precision weighting only for channels with unresolved uncertainty and lets routine channels self-regulate.

FEP's precision weighting is therefore an instance of the convergence, not an exception to it. At small scale, centralized precision allocation is viable. At scale, it converges toward a hybrid where most channels operate at locally determined precision (escalation) and higher-level precision allocation is reserved for channels where prediction errors signal genuine novelty. The two mechanisms are complementary: escalation handles routine precision; active allocation directs the bounded capacity that remains.

6.2 System M: Dupoux, LeCun, and Malik

Dupoux, LeCun, and Malik [9] propose a three-component architecture: System A (observation/world models), System B (action/planning), and System M (meta-control), where M is a fixed evolutionary routing policy that directs when A and B engage. System M monitors epistemic signals and routes data between subsystems.

This is explicitly top-down: M sits above A and B, monitoring their states and directing their activity. Theorem 3 establishes that M's monitoring itself consumes C_n proportionally to the number of subsystems and channels it oversees. As the architecture scales, more world models, more action policies, more channels, M's overhead grows, eventually crowding out the capacity for the reasoning that M is supposed to coordinate.

Their architecture would benefit from complementing top-down routing with escalation: subsystems A and B signal M only on failure (when local uncertainty exceeds a threshold), rather than M continuously polling all subsystems for state. Under this hybrid, M retains authority for genuinely novel coordination but engages only when a subsystem cannot resolve its own uncertainty, so the overhead at M's level is proportional to the *failure rate*, not the *system size*. Escalation is what keeps M viable as the architecture scales.

6.3 Global Workspace Theory

Global Workspace Theory [3, 8] already describes what is essentially an escalation architecture: specialized local processors handle routine computation autonomously, and the global workspace is engaged only when local processing fails or when integration across modules is required. Broadcasting to the global workspace is the cognitive analogue of escalation to L_n.

Theorem 3 provides the formal grounding that GWT assumes architecturally. GWT *asserts* that local processors handle routine work and the global workspace handles exceptions; the escalation policy (Definition 4) identifies the cost-minimizing criterion from the monotone cost structure, and Theorem 3 *proves* why the global workspace requires escalation to remain viable. The $O(k \cdot \alpha)$ overhead is the formal reason why global workspace engagement must be the exception rather than the rule: escalation keeps k bounded so the workspace retains capacity for genuine integration.

6.4 System 1 and System 2

Kahneman's System 1/System 2 framework [13] is the popular binary version of the encoding hierarchy. System 1 (fast, automatic, low-cost) corresponds to lower levels of $E(c, r)$; System 2 (slow, deliberate, high-cost) corresponds to L_n.

The continuous parameterization $E(c, r)$ corrects the binary discretization. There are not two systems but a continuous space of encodings with substrate-determined interfaces. A motor reflex ($c \approx 0$, $r \approx 0$) and a well-practiced heuristic (c low, r moderate) are both "System 1" in Kahneman's taxonomy, but they occupy different regions of $E(c, r)$ with different costs and different revisability. The binary collapses this structure into a distinction that, while useful for popular exposition, obscures the continuous nature of encoding and the multiplicity of interfaces.

The escalation policy also corrects the implicit control assumption: System 2 does not "decide" when to override System 1. Rather, System 1 (lower levels) handles problems autonomously, and System 2 (L_n) engages when System 1 fails. The direction of control is bottom-up, not top-down.

6.5 Automatic and Controlled Processing

Shiffrin and Schneider [18] established the foundational distinction between automatic processing (fast, parallel, low-effort, not limited by capacity) and controlled processing (slow, serial, effortful, capacity-limited). Automatic processing corresponds to actions encoded at lower levels of $E(c, r)$; controlled processing corresponds to L_n. Their key empirical finding—that practice converts controlled processing into automatic processing—is exactly the promotion mechanism of the encoding hierarchy: repeated successful execution at L_n promotes an action to lower encoding levels, freeing C_n for novel problems.

Anderson's ACT-R [1] formalizes this as proceduralization: declarative knowledge (propositions in K loaded into active context for deliberate use) compiles into procedural knowledge (condition-action rules that fire without loading K). In the present framework, proceduralization is movement along $E(c, r)$ from high-cost, high-reversibility regions toward low-cost, low-reversibility regions. ACT-R's activation-based retrieval—where propositions compete for limited working memory based on recency, frequency, and contextual relevance—is a specific implementation of contextual retrieval under bounded C_n.

Both frameworks describe the empirical trajectory that escalation predicts: as entities scale their action repertoire, an increasing fraction of actions must operate automatically (at lower encoding levels) because controlled processing at L_n cannot supervise them all. The convergence result (Theorem 3) provides the formal reason: the $O(k \cdot \alpha)$ overhead forces this transition regardless of substrate.

6.6 Control Theory: Requisite Variety and the Good Regulator

Three results from control theory provide formal antecedents for the convergence result.

Ashby's Law of Requisite Variety [2] establishes that a controller must have at least as many distinct responses as the disturbances it faces. Under bounded C_n, the regulator at L_n has variety bounded by C_n. When the system's variety—the number of distinct situations requiring distinct responses—exceeds C_n, no single regulator at L_n can achieve requisite variety. The only solution is to distribute the regulatory function across encoding levels, each regulating its local domain, with L_n regulating only the residual that lower levels cannot handle. This distribution is escalation. Escalation is how a bounded regulator achieves requisite variety over an unbounded system: not by expanding the regulator's capacity (bounded by substrate) but by distributing regulation to levels where local variety is manageable.

The Conant-Ashby Good Regulator Theorem [6] proves that every good regulator of a system must contain a model of that system. Under top-down allocation, the delegation state (Definition 3—specification, expected return, integration context) *is* this model: it is L_n's representation of each subprocess it regulates. The good regulator theorem establishes that this overhead is not a design inefficiency but a *theorem-mandated cost*: effective regulation requires the model, and the model requires context. The $O(k \cdot \alpha)$ overhead of Theorem 3 is therefore irreducible for the supervised fraction of A. This strengthens the convergence argument: the overhead cannot be optimized away because it is the minimum model that the good regulator theorem demands.

Witsenhausen's counterexample [20] demonstrates that optimal decentralized control, where multiple controllers operate with different information, can be fundamentally harder than centralized control. Escalation is a decentralized architecture: each level controls with local information. In Witsenhausen's framework, this sacrifices the optimality that a centralized controller with full

information could achieve. But bounded C_n rules out the centralized option at scale: the good regulator theorem requires a model that exceeds C_n. The relevant comparison is not between escalation and an ideal centralized controller but between escalation and a centralized controller that has overflowed its context and can no longer regulate effectively. Survival does not require Witsenhausen-optimal control. It requires a viable architecture, and escalation is the viable architecture when the centralized alternative has been eliminated by bounded context.

6.7 Subsumption Architecture

Brooks's subsumption architecture [4] is the engineering instantiation of the convergence result. In subsumption, lower-level behavioral layers (obstacle avoidance, wandering) run autonomously and continuously. Higher layers (exploration, goal-seeking) can subsume lower-layer outputs when they have something to contribute, but do not monitor or schedule lower layers during routine operation. The control direction is bottom-up: lower layers act by default, higher layers intervene on exception.

This is escalation. Each behavioral layer handles its local domain without supervisory overhead from above; higher layers engage only when lower layers produce outcomes that require coordination or override. Brooks arrived at this architecture empirically, by observing that centralized planners could not scale to real-time robotic control. Theorem 3 derives the same conclusion formally: the $O(k \cdot \alpha)$ overhead of centralized supervision exceeds C_n at scale, forcing the system toward distributed local control with higher-level exception handling.

The subsumption architecture also illustrates the one-directionality of the convergence. Brooks did not begin with escalation and converge toward centralized planning. He began with centralized planning (the STRIPS tradition), found it unscalable, and replaced it with subsumption. The reverse trajectory—starting from subsumption and converging toward centralized planning—has not occurred in the robotics literature, consistent with the irreversibility of escalation (Theorem 3).

6.8 Resource Rationality

Griffiths, Lieder, and Goodman [12] establish that the optimal algorithm for a cognitive task depends on the computational resources available: resource rationality. The optimal unconstrained algorithm may be suboptimal under resource limitations.

Theorem 3 is precisely an instance of this principle. Top-down allocation may be the optimal control architecture when C_n is unbounded: with infinite active context, the overhead of maintaining delegation state for all subprocesses is negligible, and the benefit of optimal routing is maximal. Under bounded C_n, the overhead is no longer negligible, and escalation becomes necessary to conserve the scarce resource (active context) for genuine inference rather than spending it on coordination.

Resource rationality provides the general principle; Theorem 3 provides the specific result for the control architecture domain: the resource-rational control architecture under bounded context is a hybrid where escalation handles routine problems and top-down allocation addresses the residual that only L_n can resolve.

7 Conclusion

We have shown that any control architecture converges toward escalation as the action repertoire $|A|$ grows under bounded context C_n. Top-down allocation is viable at small scale, when concurrent demands fit within C_n. But intelligence requires $|A| \gg C_n$, and the convergence is forced by three steps:

1. The escalation policy (Definition 4) is cost-minimizing under monotone encoding cost: handle each action at the lowest sufficient level, escalate on failure. This is immediate from the cost structure.
2. Theorem 3 establishes that top-down tracking incurs $O(k \cdot \alpha)$ context overhead, and that reducing k by shedding supervisory tracking of routine actions is the only recovery under bounded C_n. The shed fraction operates under escalation.
3. The crossover analysis (Section 4.1) locates the threshold at $k^* = C_n/2\alpha$—2–3 concurrent delegations for biological working memory. Beyond it, any additional concurrent demand must be handled by escalation or inference quality degrades.

The convergence is one-directional. As top-down becomes more autonomous at lower levels, it converges toward escalation. Escalation never converges toward top-down, because it never introduces anticipatory delegation context. The endpoint is a hybrid: escalation handles the routine; top-down handles the novel. Capability growth, under this hybrid, *relieves* the highest level rather than burdening it.

The falsifiable content of this result is the *cost prediction*, not the label. The claim is not that every efficient architecture is "really escalation"—which would be unfalsifiable by definitional absorption. The claim is that top-down tracking costs $O(k \cdot \alpha)$ per delegation cycle, and that k grows with $|A|$ under positive novelty rate (Section 4.1). A system that maintains top-down tracking of all actions at scale without paying $O(k \cdot \alpha)$ overhead would falsify the theorem.

This result has implications for both cognitive science and AI architecture. In cognitive science, it provides the formal grounding for Global Workspace Theory's architectural assumptions and corrects the implicit top-down control assumption in System 1/System 2 accounts. In AI architecture, it explains the trajectory from dense attention toward sparse attention, mixture-of-experts, and retrieval-augmented generation: this trajectory is not a design trend but the scaling convergence the theorem predicts. Systems scaling beyond k^* *must* adopt escalation-like structure or degrade.

The series to this point has established two results: knowledge under bounded context necessarily forms a graph [15]; and the control architecture over that graph converges toward escalation as capability grows under bounded context (this paper). Together, these results constrain the architecture of any system that accumulates knowledge, acts under uncertainty, and operates within bounded active context—which is every physical system that persists.

Acknowledgments

The author used Claude (Anthropic) for drafting assistance, literature review, and LaTeX preparation. All theoretical content, proofs, and arguments are the author's own.

References

[1] J. R. Anderson. *Rules of the Mind*. Lawrence Erlbaum Associates, 1993.

[2] W. R. Ashby. *An Introduction to Cybernetics*. Chapman & Hall, 1956.

[3] B. J. Baars. *A Cognitive Theory of Consciousness*. Cambridge University Press, 1988.

[4] R. A. Brooks. A robust layered control system for a mobile robot. *IEEE Journal on Robotics and Automation*, 2(1):14–23, 1986.

[5] N. Cowan. The magical number 4 in short-term memory: A reconsideration of mental storage capacity. *Behavioral and Brain Sciences*, 24(1):87–114, 2001.

[6] R. C. Conant and W. R. Ashby. Every good regulator of a system must be a model of that system. *International Journal of Systems Science*, 1(2):89–97, 1970.

[7] T. M. Cover and J. A. Thomas. *Elements of Information Theory*. Wiley-Interscience, 2nd edition, 2006.

[8] S. Dehaene. *Consciousness and the Brain: Deciphering How the Brain Codes Our Thoughts*. Viking Press, 2011.

[9] E. Dupoux, Y. LeCun, and J. Malik. Why AI systems don't learn and what to do about it: Lessons on autonomous learning from cognitive science. *arXiv preprint arXiv:2603.15381*, 2026.

[10] K. Friston. The free-energy principle: A unified brain theory? *Nature Reviews Neuroscience*, 11(2):127–138, 2010.

[11] K. Friston. Active inference and learning. *Neuroscience & Biobehavioral Reviews*, 68:862–879, 2017.

[12] T. L. Griffiths, F. Lieder, and N. D. Goodman. Rational use of cognitive resources: Levels of analysis between the computational and the algorithmic. *Topics in Cognitive Science*, 7(2):217–229, 2015.

[13] D. Kahneman. *Thinking, Fast and Slow*. Farrar, Straus and Giroux, 2011.

[14] N. F. Liu, K. Lin, J. Hewitt, A. Paranjape, M. Bevilacqua, F. Petroni, and P. Liang. Lost in the middle: How language models use long contexts. *Transactions of the Association for Computational Linguistics*, 12:157–173, 2024.

[15] P. D. McCarthy. Graph structure is necessary for information preservation under bounded context. Working paper, 2026.

[16] H. A. Simon. A behavioral model of rational choice. *The Quarterly Journal of Economics*, 69(1):99–118, 1955.

[17] H. A. Simon. The architecture of complexity. *Proceedings of the American Philosophical Society*, 106(6):467–482, 1962.

[18] R. M. Shiffrin and W. Schneider. Controlled and automatic human information processing: II. Perceptual learning, automatic attending and a general theory. *Psychological Review*, 84(2):127–190, 1977.

[19] O. E. Williamson. Hierarchical control and optimum firm size. *Journal of Political Economy*, 75(2):123–138, 1967.

[20] H. S. Witsenhausen. A counterexample in stochastic optimum control. *SIAM Journal on Control*, 6(1):131–147, 1968.